

Scan Vision's Core

MIRAGE 2.0 — Reverse Engineering — Official Writeup

CATEGORY	FILES	DEPLOYMENT
Reverse Engineering	vision_core (stripped)	Offline binary — run locally
VULNERABILITY	STATUS	
N/A — embedded-constant XOR comparison	Solved	

Challenge Premise

“Vision's core has gone dark. The diagnostic interface still responds, but the authentication routine protecting the recovery channel is buried inside the damaged core. Recover the diagnostic password and restore the core.” The binary is stripped, with no anti-debug measures of any kind — the simplest reversing exercise in this set.

Reversing the Binary

The program prints a banner and prompts “Diagnostic password: ”, reading a line via `fgetc`s into an 80-byte buffer. A single non-trivial function (found by tracing what `main` calls after the prompt) does the following:

- Calls `strlen()` on the input and requires it to be exactly 10 characters — anything else fails immediately with “authentication rejected”.
- Loops over the 10 input bytes, XORing each with the constant `0x27` and comparing the result against a fixed 10-byte table stored in `.rodata`: `71 16 74 6e 68 69 04 13 15 06`.

Recovering the password is simply XOR-ing that table with `0x27` byte-by-byte:

```
>>> table = bytes.fromhex('71 16 74 6e 68 69 04 13 15 06'.replace(' ', ''))
>>> bytes(b ^ 0x27 for b in table)
b'V1SI0N#42!'
```

Participant Solve Steps

```
$ ./vision_core
Diagnostic password: V1SI0N#42!
```

FLAG

```
MIRAGE{V1S10N_C0R3!_R3ST0R3D#42}
```

Verification

The flag string also sits in plaintext inside `.rodata` at a fixed offset, independently confirming the recovered value without even needing to trace the password logic. Piping `V1SION#42!` into the live binary printed “diagnostic authentication accepted” / “recovery channel restored” followed by the flag shown above.